

Graph Problems Related to Gate Matrix Layout and PLA Folding

Rolf H. Möhring*, Berlin

Abstract — Zusammenfassung

Graph Problems Related to Gate Matrix Layout and PLA Folding. This paper gives a survey on graph problems occurring in linear VLSI layout architectures such as gate matrix layout, folding of programmable logic arrays, and Weinberger arrays. These include a variety of mostly independently investigated graph problems such as augmentation of a given graph to an interval graph with small clique size, node search of graphs, matching problems with side constraints, and other. We discuss implications of graph theoretic results for the VLSI layout problems and survey new research directions. New results presented include NP-hardness of gate matrix layout on chordal graphs, efficient algorithms for trees, cographs, and certain chordal graphs, Lagrangean relaxation and approximation algorithms based on on-line interval graph augmentation.

Key words: linear VLSI layout architectures, gate matrix layout, PLA folding, interval graph augmentation, graph search, path width, vertex separation, complexity, approximation algorithms, matching with side constraints.

Graphentheoretische Probleme beim Gate Matrix Layout und PLA Folding. Der Artikel behandelt graphentheoretische Probleme, die bei linearen VLSI-Layout Architekturen wie Gate-Matrix-Layout, Programmierbaren Logischen Arrays und Weinberger Arrays auftreten. Zu diesen gehören u.a. die Einbettung von Graphen in Intervallgraphen mit kleiner Cliquengröße, Suchspiele auf Graphen, und Zuordnungsprobleme mit Nebenbedingungen. Wir diskutieren Folgerungen aus graphentheoretischen Ergebnissen für das VLSI Layout und geben eine Übersicht über neue Forschungsergebnisse. Hierzu gehören u.a.: die NP-Vollständigkeit des Gate-Matrix-Layout auf chordalen Graphen, effiziente Algorithmen für Bäume, Cographen sowie gewisse chordale Graphen, Langrange Relaxation und approximierende Algorithmen, die auf der On-line Konstruktion von Intervallgrapheinbettungen basieren.

1. Introduction

We study a class of graph-theoretic problems that arise in certain linear VLSI layout problems such as Weinberger arrays, gate matrix layout and PLA-folding.

The term *linear* refers to the fact that the most important degrees of freedom in the underlying VLSI architecture consist of linear (i.e. one-dimensional) arrangements of the relevant physical objects, the *gates*.

* Technical University of Berlin, Fachbereich Mathematik, Straße des 17. Juni 136, 1000 Berlin 12. This work was supported by the Deutsche Forschungsgemeinschaft (DFG).

In the most general form, an instance of such a linear layout problem consists of a $m \times n$ 1-0 matrix $M = (m_{ij})$ (the *net-gate matrix*), whose rows and columns represent the nets $N_1, \dots, N_m$ and the gates $G_1, \dots, G_n$ of the circuit, respectively.

The gates may be thought of as the basic electronic devices that are arranged linearly in a row, and the nets as realizing connections between them (details are given in Section 2). Net N_i must connect all gates G_j with $m_{ij} = 1$. Connections are realized rowwise by reserving for a given permutation of the gates (columns) for every net N_i the part of the row from the leftmost to the rightmost gate to which a connection must be established.

This can be expressed more formally by considering for a permutation π of the gates the *augmented net-gate matrix* $M_\pi = (\bar{m}_{ij})$ with

$$\bar{m}_{ij} := \begin{cases} 1 & \text{if there are Gates } G_r, G_s \text{ with} \\ & \pi(r) \leq j \leq \pi(s) \text{ and } m_{ir} = m_{is} = 1 \\ 0 & \text{otherwise} \end{cases}$$

Nets of the augmented net-gate matrix may share the same row (called *track*) if they have no gate in common. An assignment of augmented nets to tracks preserving this property is called a *feasible track assignment*.

The additional ones in M (with respect to the same column permutation of M) are called *fill-ins*. They are represented by a “*” to distinguish them from the given ones in M (see Figure 1.1 below).

The result of a permutation of the gates (*gate arrangement*) and an associated feasible track assignment is called a *layout*. Its area is proportional to $(\# \text{ gates}) \cdot (\# \text{ tracks}) = n \cdot (\# \text{ tracks})$.

So constructing an area-minimal layout (*optimal layout*) is equivalent to finding a gate arrangement and an associated feasible track assignment such that the number of tracks is minimum.

In matrix terminology, this leads to the following *matrix permutation problem* (MPP):

Given: A 0-1 matrix (net-gate matrix) M .

Problem: Find a permutation of the columns and an assignment of the augmented rows (nets) to tracks such that the number of tracks is minimum.

An example is given in Figure 1.1. We denote the minimum number of tracks by $t(M)$ and call a layout with $t(M)$ tracks an *optimal layout*. Due to the mentioned origin of this problem in VLSI-applications, there is an enormous body of papers on it. This article gives an overview on available results and new developments.

Section 2 deals with the VLSI background and models the linear VLSI layout technologies “Weinberger arrays”, “gate matrix layout” and “PLA folding” as instances of the general MPP. Some of these applications lead to restricted MPP’s in the sense that either the permutation of the gates (e.g. fixed first and last gate) or the assignment of nets to tracks (e.g. at most two per track) are restricted.

	G ₁	G ₂	G ₃	G ₄
N ₁	1	1		
N ₂	1		1	
N ₃		1		1
N ₄			1	1
N ₅			1	

net - gate matrix

	G ₁	G ₂	G ₄	G ₃
N ₁	1	1		
N ₂	1	*	*	1
N ₃		1	1	
N ₄			1	1
N ₅				1

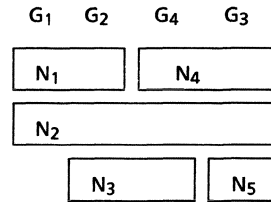
augmented net - gate matrix
for $\pi = (1, 2, 4, 3)$ associated feasible
track assignment

Figure 1.1. An example MPP

Section 3 gives several equivalent graph theoretic problems with different and partly independent background. Among them are: augmentation of a graph to an interval graph with small clique size, a node search problem, determining the path-width of a graph, matching problems with side constraints and others.

Section 4 is devoted to the complexity of the problem, and to reductions between some of the specialized versions. The general problem is already NP-hard on chordal graphs, but solvable in polynomial time on trees and cographs and if the number of tracks is fixed. Sharper version of NP-completeness results are also obtained for certain variants of PLA folding.

Finally, Section 5 deals with exact and approximation algorithms for solving MPP's. Due to the practical relevance of the problem, many heuristics have been proposed in the literature. Nevertheless, the problem of the existence of an approximation algorithm with constant relative performance bound has remained open. We present a class of on-line algorithms based on incremental interval graph generation that may be promising in this respect.

Our graph-theoretic notation is usually standard. For all notions and definitions not explicitly stated here, we refer to [Go80] or [Ev79].

2. The VLSI Background

The matrix permutation problem is typical for a number of “regular” layout styles for the generation of random logic modules in VLSI.

Such a random logic module may be seen as an irregular structure of basic components such as transistors, gates, flip-flops etc. It is given by some input description, e.g., by a transistor scheme, a logic scheme, or a set of Boolean functions. From this description, a concrete layout (a physical module) must be constructed according to some layout architecture style. A *regular* layout style is a style in which basic topological relationships between the physical components on the chip area are known in advance (e.g. restricted placement, predefined locations for the arrangements of gates etc.).

Typical such layout styles are *Weinberger arrays*, *gate matrix layout* and *programmable logic arrays* discussed below, see also [GL88] and [BMHS84].

Weinberger Array. Weinberger arrays were introduced in [Wei67] as a layout architecture for Boolean functions that are given by a circuit consisting only of NOR-gates (see Figure 2.1). Each NOR-gate is converted into an nMOS gate (i.e. a gate in the nMOS VLSI technology, see e.g. [MC80] for details); and these gates are arranged in a linear array that constitute the columns of an associated MPP (see Figure 2.2).

Each column of this MPP consists of two vertical wires. One wire is connected to the pull-up transistor and serves as the output port, while the other wire is connected with the ground power line. (Usually, two neighboring gates share a common ground wire.) The input signals to the gates are obtained from horizontal polysilicon intervals on a row. A transistor is formed by the intersection of an extension of such a polysilicon interval with a diffusion segment between the output the ground lines. For example, transistors a and b are formed by connecting row 1 and 2 to diffusion segments in gate A. The output of gate A is connected to the last row which serves as input to gate F, etc.

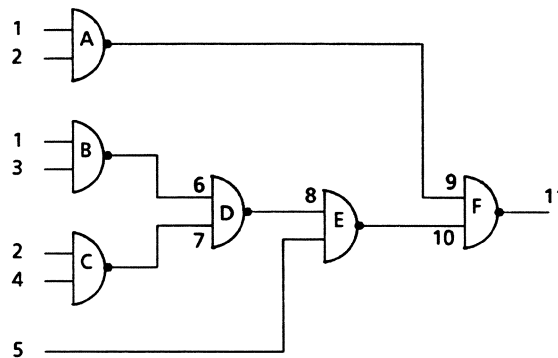


Figure 2.1. A circuit of NOR gates

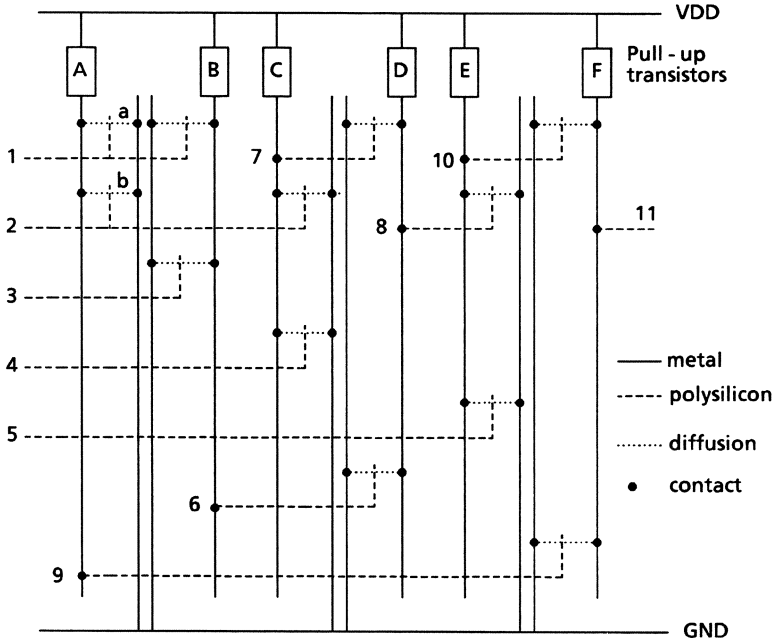


Figure 2.2. A Weinberger array layout of the circuit of Figure 2.1

Note that connections (i.e. polysilicon intervals) may be placed on the same line if they do not overlap. Since the number of gates is fixed, minimization of the layout area is equivalent to reducing the number of rows, i.e. by finding a suitable permutation of the gates (which defines the length of the polysilicon intervals) and an associated row assignment of the intervals such that the number of rows is minimum.

So we obtain the following *Weinberger MPP* (WMPP):

- Given:
- A collection $G_0, G_1, \dots, G_n, G_{n+1}$ of gates, where $G_1, \dots, G_n$ represent the NOR gates of the circuit, and where G_0 and G_{n+1} represent the input (on the left) and output signals (on the right) of the circuit, respectively.
 - A collection of nets $N_1, \dots, N_m$. Each net N_i consists of those gates G_j , to which it is output or input.
 - The net-gate matrix M .

Problem: Find a permutation of $G_1, \dots, G_n$ (i.e. the positions of G_0 and G_{n+1} are fixed) and an associated feasible track assignment (layout) such that the number of tracks is minimum.

Figure 2.3 shows the matrix M , a layout corresponding to Figure 2.2 and an optimal layout for the above example.

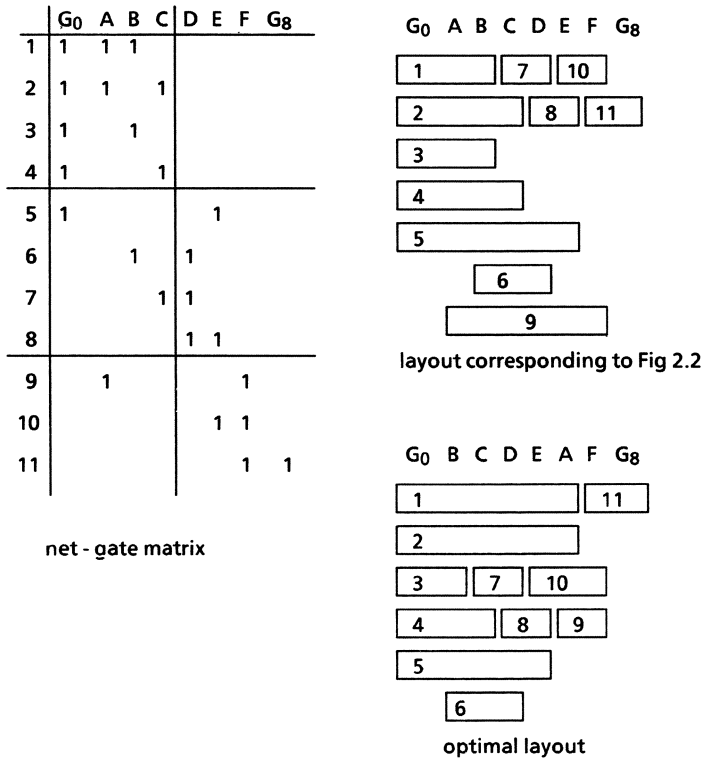


Figure 2.3. Net-gate matrix and two layouts for the WMPP of Figure 2.1

Gate Matrix Layout. This architecture was introduced by [LLa80] as a regular layout style for large scale transistor circuits in the CMOS technology. In such a layout, a vertical polysilicon wire corresponding to an input, internal or output signal is placed in every column (columns *A*, *B*, *C*, *D*, *E*, *F*, *G* and *Z* in Figure 2.4). All transistors using the same signal are constructed along the same column (e.g. transistors 1 and 7 in column *A* of Figure 2.4). Connections among transistors are made by horizontal metal lines, while connections to *Vdd/Vss* are in a second metal layer (and irrelevant to the underlying *MPP*). They are indicated by up and downward arrows in Figure 2.4.

A *net* is a collection of metal lines and transistors to which it must be connected. Net N_1 in row *A* of Figure 2.4 spans from column *A* to *G* and is connected to three transistors (1, 2, and 3) and one metal line (*G*). The (slightly simplified) assumption about the realization of nets is that the series-parallel transistor circuit of each net and its output signals can be realized in a row regardless the permutation of the metal lines and transistors.

As with Weinberger arrays, minimizing the layout area leads to the following *gate matrix permutation problem* (GMPP).

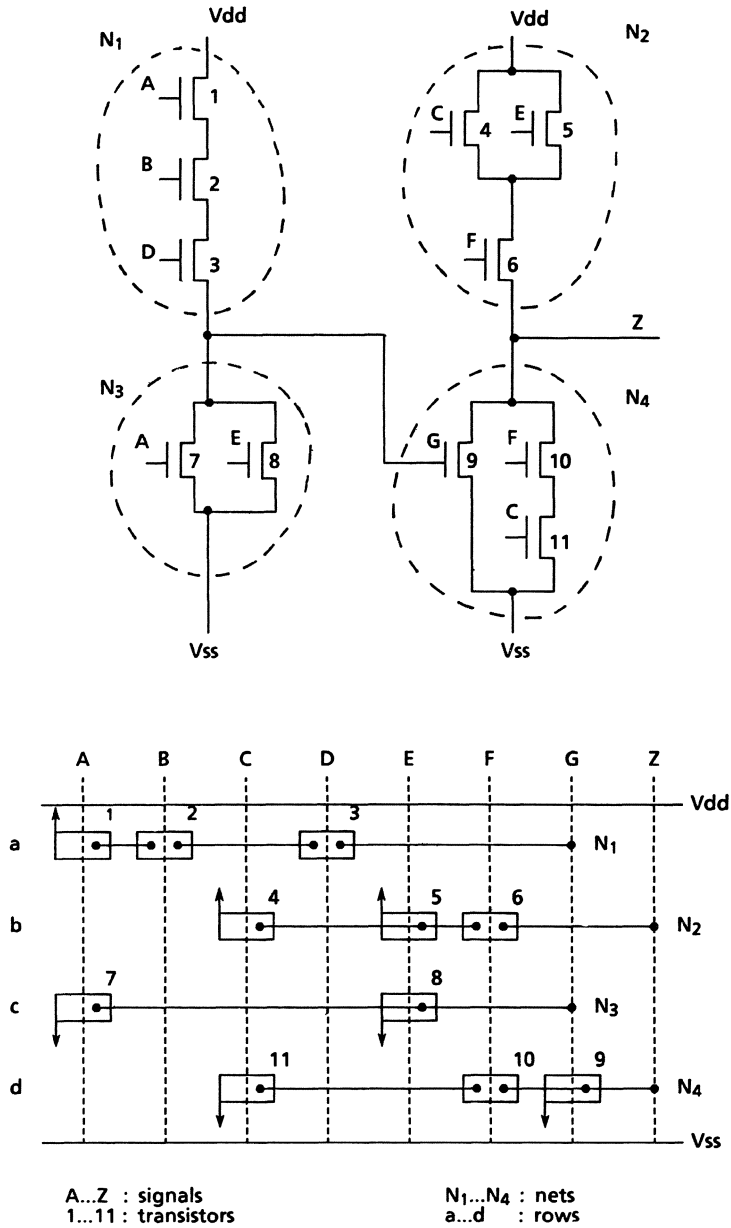


Figure 2.4. A transistor circuit and an associated gate matrix layout

- Given:
- A collection $G_1, G_2, \dots, G_n$ of gates representing metal lines or transistors of a gate matrix.
 - A collection $N_1, \dots, N_m$ of nets, where each N_i is a subset of $\{G_1, \dots, G_n\}$.
 - The net-gate matrix M .

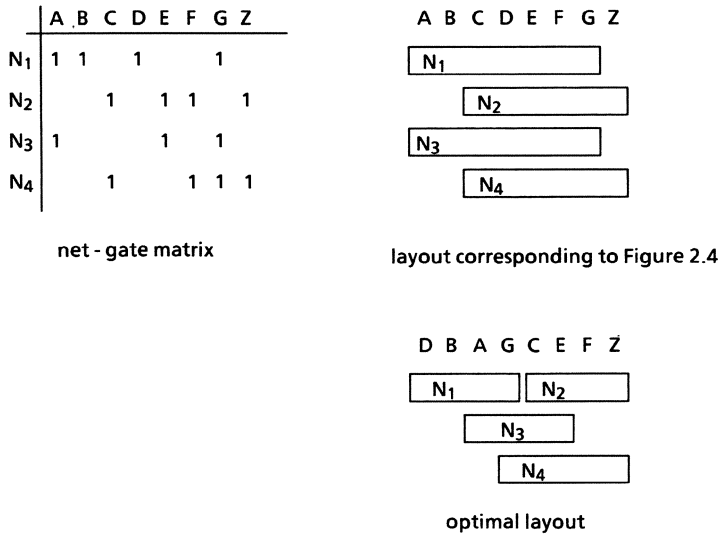


Figure 2.5. Net-gate matrix and two layouts for the GAMP of Figure 2.4

Problem: Find a permutation of $G_1, \dots, G_n$ and an associated feasible track assignment (layout) such that the number of tracks is minimum.

Figure 2.5 displays the net-gate matrix and two layouts for the above example.

The GMMP is the combinatorial core of the problem to construct an area minimal gate matrix layout. Additional (and usually neglected) features are 1) that one may distinguish two collections of rows (p -devices and n -devices) that permit independent column permutations, and 2) that a net may require more than one row depending on the permutation of its gates. While this second feature can be modeled within the *MPP* formulation by appropriate net splitting, incorporation of the first feature may lead to better layouts [NFKY86]. For further technical information we refer to [SM83], [WHW85].

Programmable Logic Array. A programmable logic array (PLA) realizes a collection of Boolean functions given in disjunctive form (two-level sum of product form) on a two-dimensional array (see Figure 2.6).

This array consists of an AND-plane and an OR-plane. For every variable x_i of the Boolean functions, there is an input signal to the AND-plane (in fact, both inputs x_i and $\bar{x}_i$ are generated). Each row of the AND-plane produces a term that is an input to the OR-plane. The columns of the OR-plane correspond to the different Boolean functions and combine the appropriate product terms by an OR operation. By adding storage elements and simple feedback connections, a PLA can very easily be used to implement a sequential circuit. This application of PLA's is popular in the design of microcontrollers. For further technical information, we refer to [FM75].

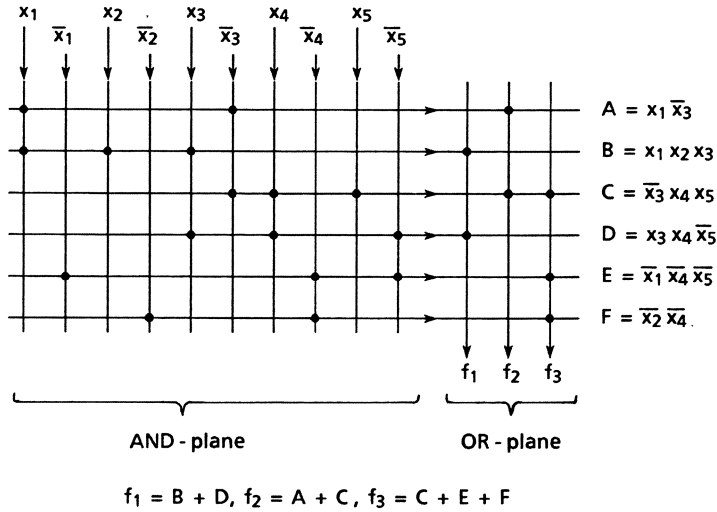


Figure 2.6. A PLA layout of 3 Boolean functions

For reducing the area of a PLA, two techniques can be applied. *Logic minimization* for reducing the number of rows (= product terms) and *PLA folding* for reducing the number of columns. The first technique is the same as finding the minimum number of prime implicants for a set of Boolean functions (see e.g. [BMHS84]). It is usually applied before the folding.

The folding allows two (sometimes also more) signals to share a row (in the AND-plane) or a column (in the OR-plane). This leads to the same class of MPP's in both the AND and the OR-plane.

PLAMPP

Given:

- A collection of gates $G_1, \dots, G_n$ that correspond to the signals in one plane of a PLA.
- A collection of nets $N_1, \dots, N_m$, where each net is a set of signals that have to be combined by AND (in the AND-plane) or OR (in the OR-plane).
- The net-gate matrix M .

Problem: Find a permutation of the gates $G_1, \dots, G_n$ and a feasible assignment of at most two nets to a track (*PLA layout*) such that the number of tracks is minimum.

Figure 2.7 displays the net-gate matrix and several layouts for the example of Figure 2.6.

There are several more restrictive versions of the PLA folding problem. If gates occurring in the second net of a track may not occur in the first net of a track with two nets, one speaks of *block folding*. In that case, any folding defines a partition

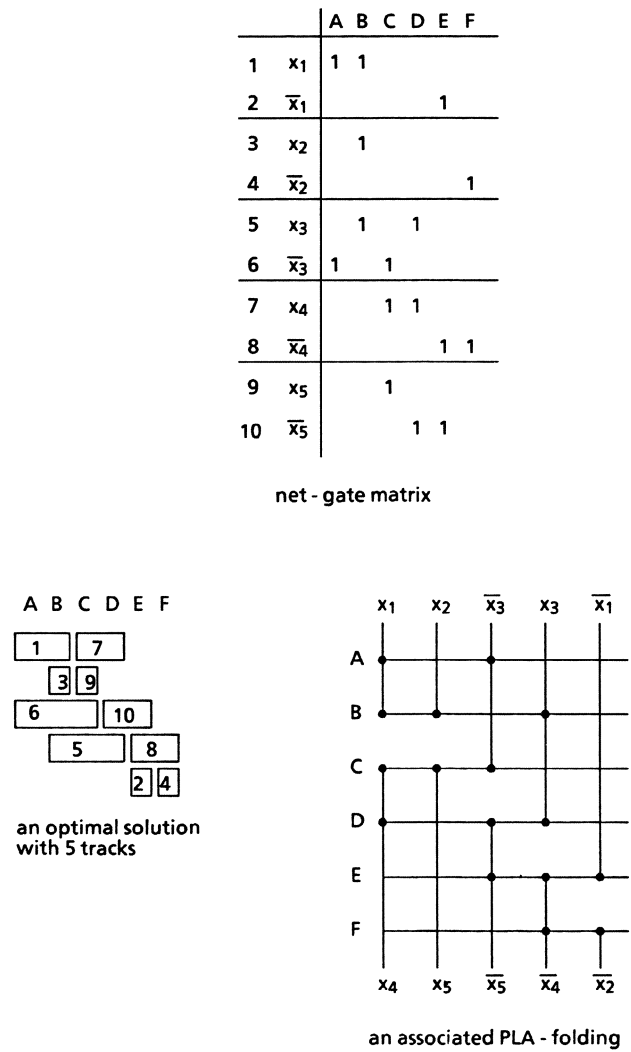
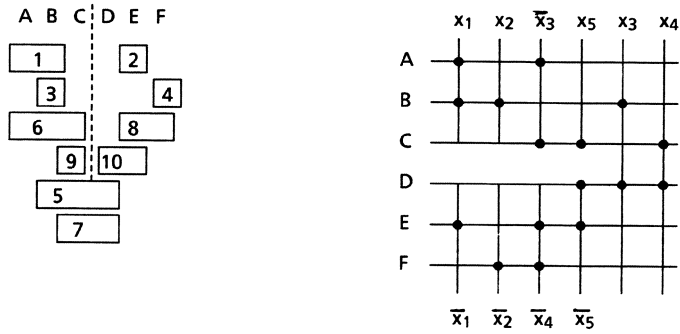


Figure 2.7. The net-gate matrix and an optimal layout for the AND-plane of Figure 2.6

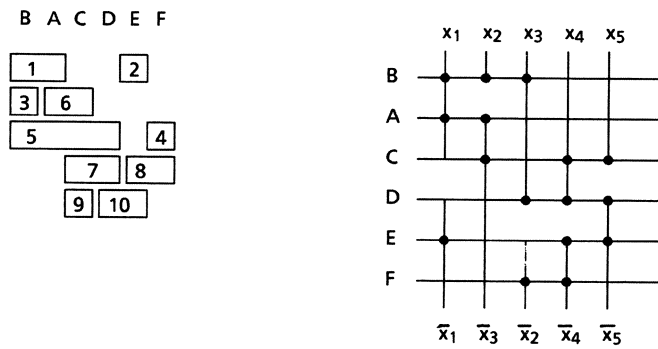
G_1, G_2 of the gates such that if net N_i is before N_j in the same track, then $N_i \in G_1$ and $N_j \in G_2$.

If the nets are pre-assigned to the two sides of the layout, one speaks of *constrained folding*. In that case, a partition N_1, N_2 of the nets is given as input to the problem, and any restricted folding may only assign net N_i before N_j on the same track if $N_i \in N_1$ and $N_j \in N_2$.

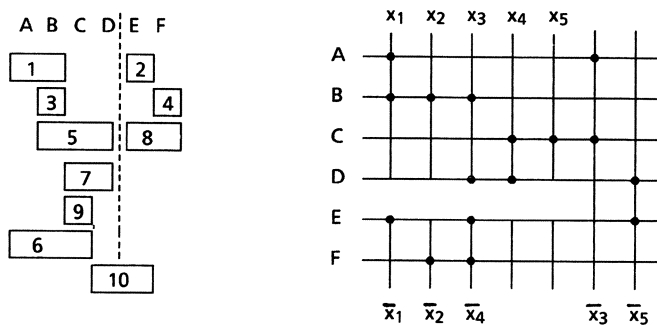
The combination of block folding and restricted folding is called *constrained block folding*. Figure 2.8 shows optimal layouts for these different folding problems for the example of Figure 2.7.



a) optimal block folding and associated PLA with 6 tracks



b) optimal constrained folding and associated PLA for the net partition $\{1,3,5,7,9\}, \{2,4,6,8,10\}$ with 5 tracks



c) optimal constrained block folding and associated PLA for the partition from b) with 7 tracks

Figure 2.8. Restricted PLA-foldings

3. Graph-Theoretic Formulations and Related Problems

We will now consider several graph theoretic problems that are equivalent to the VLSI layout problems discussed in the previous sections. These problems have their own graph theoretic background and have to a large extent been investigated independently of each other.

Interval graph augmentation. This formulation occurs already in the first papers on gate matrix layout [Win82], [Win83]. For Weinberger arrays, similar considerations are made in [OMKK79].

Let V be a finite set and $(I_v)_{v \in V}$ be a collection of (not necessarily distinct) intervals I_v of a linearly ordered universe (such as the real line or a permutation of the gates). Such a collection of intervals $(I_v)_{v \in V}$ defines a partial order $P = (V, <)$ on V by putting

$$(3.1) \quad u < v \Leftrightarrow I_u \text{ is entirely to the left of } I_v.$$

It also defines an undirected graph $G = (V, E)$ on V by putting

$$(3.2) \quad (u, v) \in E \Leftrightarrow I_u \text{ and } I_v \text{ intersect (i.e. } I_u \cap I_v \neq \emptyset)$$

A partial order P and a graph G obtained in that way are called an *interval order* and an *interval graph*, respectively, and an associated collection of intervals $(I_v)_{v \in V}$ is called an *interval representation* of P or G . An example is given in Figure 3.1.

Interval orders and interval graphs model the sequential and intersection structure of a set of intervals of the real line. This is why they have many applications dealing with intersection and consecutiveness such as the gene structure in molecular biology, seriation in archeology, preference and indifference relations in measurement theory, and consecutive retrieval, VLSI channel routing, and gate matrix layout in computer science. For more information on these applications, see [Go80], [Go85], [Mö85], [Mö89].

Note that different interval representations with the same intersection behavior define the same interval graph but possibly different interval orders. We call all interval orders related in this way to a fixed interval graph G the interval orders *associated* with G .

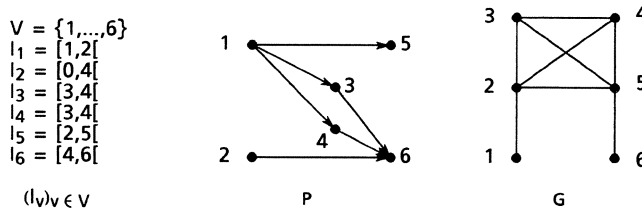


Figure 3.1. A collecting of intervals with associated interval order P (as transitively reduced directed acyclic graph) and interval graph G

Consider now an MPP with net-gate matrix M . This matrix M defines a graph $G = ((V(G), E(G)))$ by taking the *intersection graph* of the rows, i.e., $V(G)$ is the set of nets (rows), and two nets are connected by an edge if they share a gate (i.e. there is a column with 1's in both rows). G is called the *net adjacency graph* [DKL87], or *incompatibility graph* [ALN88] since its edges (u, v) express that the nets u, v cannot be assigned the same track in a feasible layout.

For any gate permutation π of M , the associated augmented matrix M_π defines a collection of intervals (the augmented nets) of the linear order $G_{i_1}, \dots, G_{i_n}$ defined by π on the gates. This collection of intervals defines an interval graph $H = (V(H), E(H))$ that contains G in the sense that $V(G) = V(H)$ and $E(G) \subseteq E(H)$, i.e. by *augmenting* the edge set of G . (This follows directly from the fact that two nets that share a gate in M share also a gate in M_π .) A feasible track assignment for M_π corresponds then obviously to a *coloring* of H , in which tracks correspond to color classes.

This shows that every feasible layout for M induces a coloring of an interval graph augmentation of G , the incompatibility graph of M . The converse is also true as the following lemma shows.

3.1 Lemma: *Let $G = (V(G), E(G))$ be the incompatibility graph of a net-gate matrix M . Let $H = (V(H), E(H))$ be an interval graph augmentation of G and let $P = (V(H), <)$ be any interval order associated with H . Then:*

- a) P induces a partial order on the gates $G_1, \dots, G_n$ of M by putting

$$(3.3) \quad G_r < G_s \text{ if there are nets } N_i \neq N_j \text{ incident to } G_r \text{ and } G_s, \\ \text{respectively (i.e. } m_{ir} = m_{js} = 1), \text{ and } N_i < N_j \text{ in } P.$$
- b) Any linear extension $G_{j_1}, \dots, G_{j_n}$ of the gate order of a) induces a permutation π such that the augmented net-gate matrix M_π is an interval representation of H and P .
- c) Any coloring of H induces a track assignment of M_π by taking each color class as a track and ordering the nets of the color class according to P .

The proof is straightforward and left to the reader. An example of these constructions is presented in Figure 3.2. An immediate consequence of these consideration is:

3.2 Theorem: *The minimum number of tracks of a feasible layout for a net-gate matrix M is equal to the smallest chromatic number of an interval graph augmentation H of the incompatibility graph G of M , i.e.*

$$(3.4) \quad t(M) = \min\{\chi(H) | H \text{ is an interval graph with } E(G) \subseteq E(H)\}$$

We briefly discuss the computational complexity of the constructions of Lemma 3.1. An adjacency matrix of G can be constructed from M in $O(n \cdot m^2)$ time by looking at each column separately and inserting the corresponding edge entries in the adjacency matrix.

An interval representation of an interval graph H and an associated interval order P (in interval representation) can be constructed in $O(|V(H)| + |E(H)|)$ time by

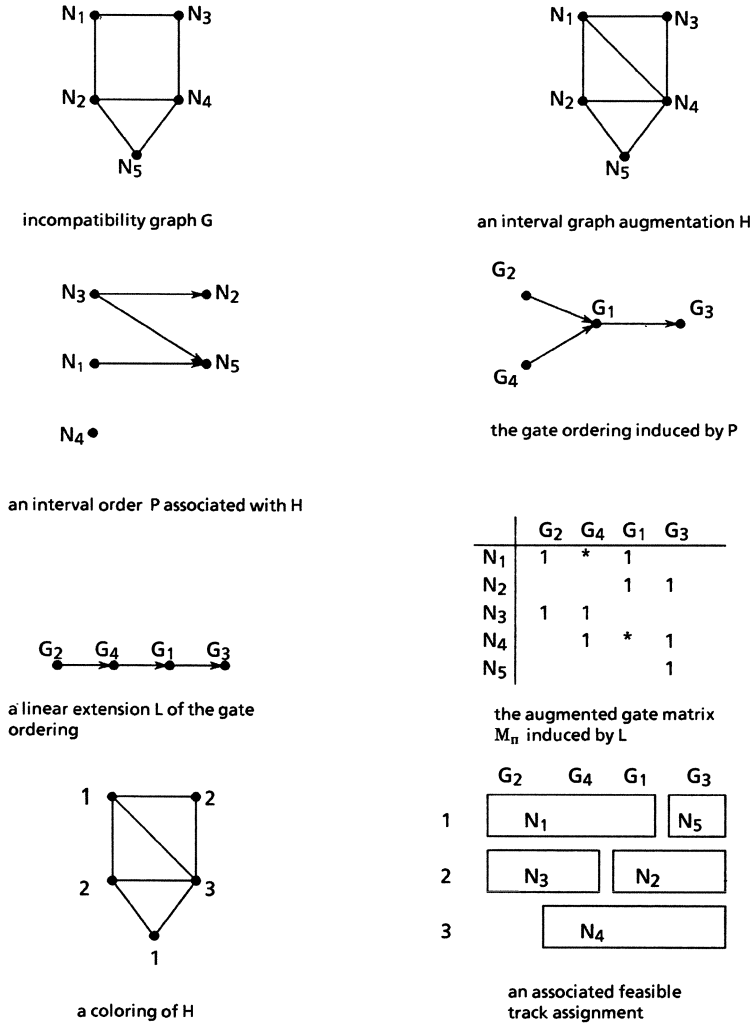


Figure 3.2. An illustration of Lemma 3.1 on the example of Figure 1.1

PQ-tree techniques [BL76, KM89]. The gate order induced by P can be obtained efficiently by scanning an interval representation of P from left to right and constructing an ordered partition of the gates as follows:

For the i -th right endpoint encountered in the scan, let the set G_i consist of all gates that are incident to nets ending (as intervals) at the current endpoint and do not belong to any previously constructed G_j . The partition $G_1, \dots, G_k$ thus constructed defines the gate ordering: all gates from G_i are incomparable among each other and precede all gates from G_{i+1} ($i = 1, \dots, k - 1$). This follows from the fact that all gates not yet considered at the current endpoint must belong to nets that come later in

the interval representation and are thus successors in P of the currently ending nets. So linear extensions of the gate ordering are just permutations within the classes G_i of the partition. It follows that the partition and a linear extension (gate permutation) can be constructed in $O(n \cdot m)$ time.

An optimal coloring of an interval graph H can be obtained in $O(|V(H)|)$ time from an interval representation by scanning the interval representation from left to right [GLL82]. When the left endpoint of an interval is encountered in the scan, the corresponding vertex of H is assigned the smallest color from the set $\{1, 2, \dots, n\}$ of colors that has not been assigned to intervals containing the current endpoint.

Obviously, the number of colors thus required is equal to the maximum number of intervals that intersect in a common point, i.e. the maximum size $\omega(H)$ of a clique of H . Since any coloring requires at least $\omega(H)$ colors, it follows that the coloring is optimal and $\chi(H) = \omega(H)$.

These arguments show that the hard core of the MPP is the construction of the right gate permutation, or, in terms of the interval graph augmentation, to find the right augmentation of the incompatibility graph to an interval graph. The track assignment problem or interval graph coloring problem can then be solved optimally by a simple linear-time algorithm.

Note that the interval graph coloring algorithm described here can of course also be directly carried out on augmented net-gate matrices (remember that they represent interval graphs). In this context, it is known as the *left-edge algorithm* (starting from the left edge of the rectangle described by the matrix) and it already occurs in channel routing applications of interval graphs [HS71]. It shows in particular that the minimum number of tracks for an augmented net-gate matrix M_π is equal to the maximum column sum of M_π .

Because of the easy solvability of a track assignment/interval graph coloring problem, we can rephrase our original problems as follows:

(MPP):

Given a 0-1 matrix M , find a permutation π of the columns of M such that the maximum column sum of the augmented matrix M_π is as small as possible.

Interval graph augmentation (IGA):

Given a graph G , find an augmentation of G to an interval graph H whose clique size $\omega(H)$ is as small as possible.

The smallest clique size $\omega(H)$ of an interval graph augmentation of G is also called the *interval thickness* of G . Because of its equivalence with MPP, we will denote it also by $t(G)$, and call it the *track number* of G .

So far, we have seen that every MPP can be transformed to an instance of IGA. The converse is, of course, also true, since every graph can be represented as the incompatibility graph of some MPP (e.g. by introducing a column for every edge of G with two 1-entries for the vertices joined by this edge).

This gives:

3.3 Theorem: *MPP and IGA are polynomially equivalent.*

The interpretation of MPP as IGA makes available the large body of algorithmic techniques for interval graphs, see e.g. [Go80], [Mö85], [Mö89]. Most of them are based on the following characterization of [FG65] of interval graphs.

3.4 Theorem: *A graph G is an interval graph iff its maximal cliques can be linearly ordered such that, for every vertex v , the maximal cliques containing v occur consecutively.*

Any such arrangement is called a *consecutive clique arrangement*. Such arrangements can be constructed and maintained by PQ-trees [BL76] or their specialized versions for interval graphs, the MPQ-trees [KM89]. These data structures will be useful for approximation algorithms discussed in Section 5.

Loosely speaking, the general idea of the interval graph augmentation approach to the MPP can be expressed as making the cliques of G consecutive (in the sense of Theorem 3.4) by extending or joining them to cliques of an interval graph while keeping the size of the new cliques small.

Path width. The path width of a graph was considered by Robertson and Seymour [RS83] in the first part of their series of papers on graph minors.

They define a *path decomposition* of a graph G as a sequence $X_1, \dots, X_r$ of subsets of $V(G)$ such that

$$(3.5) \quad \text{for every edge } e \text{ of } G, \text{ some } X_i \text{ contains both ends of } e$$

and

$$(3.6) \quad \text{for } 1 \leq i \leq j \leq l \leq r, \quad X_i \cap X_l \subseteq X_j$$

hold. The *path width* of G (denoted by $pw(G)$) is the minimum value of $k \geq 0$ such that G has a path decomposition $X_1, \dots, X_r$ with $|X_i| \leq k + 1$ ($i = 1, \dots, r$).

This notion is almost identical to interval graph augmentation. In fact, if G has a path decomposition $X_1, \dots, X_r$ with $|X_i| \leq k + 1$, then the graph H defined by letting $X_1, \dots, X_r$ be its maximal cliques is an interval graph because of (3.6) and Theorem 3.4, and fulfills $E(G) \subseteq E(H)$ because of (3.5), and $\omega(H) = \max_i |X_i| \leq k + 1$. Conversely, if H is an interval graph augmentation of G , then any consecutive clique arrangement $C_1, \dots, C_r$ of H defines a path partition of G because of Theorem 3.4, and $|C_i| \leq k + 1$ with $k = \omega(G) - 1$. This gives:

3.5 Proposition: *Determining the path width of a graph G is polynomially equivalent to IGA. In particular, $pw(G) = t(G) - 1$.*

This equivalence permits a direct translation of deep results from the Robertson-Seymour theory to gate matrix layout. The most important of these is related to the notion of the minor of a graph.

H is a *minor* of G if H can be obtained from G by deleting some vertices and/or edges, and/or contracting some edges. It is easy to see that

$$(3.7) \quad t(H) \leq t(G) \text{ if } H \text{ is a minor of } G.$$

This implies directly that for any fixed k , the class of graphs G with $t(G) \leq k$ is *closed under taking minors* (i.e., if G belongs to this class and H is a minor of G then H belongs also to this class). This is the starting point for the application of the following results of Robertson and Seymour.

3.6 Theorem [RS87]: *Let F be any set of graphs closed under minors. Then there are finitely many graphs $H_1, \dots, H_r$ such that*

$$G \in F \Leftrightarrow G \text{ does not contain } H_i \text{ as a minor, } i = 1, \dots, r.$$

This is a direct consequence of the proof of the Wagner Conjecture (no class of graphs has infinite antichains under the minor ordering) in [RS87] and the closedness property under taking minors.

3.7 Theorem [RS86]: *For any fixed graph H , it can be tested in polynomial time whether a graph G contains a minor isomorphic to H .*

The combination of these theorems yields the existence of a polynomial time algorithm for testing membership for any minor-closed family of graphs, thus in particular for the class of graphs with bounded path width.

However, since no proof of Wagner's conjecture can be entirely constructive [FRS87], Theorem 3.6 is a pure *existence result* for the finite family of forbidden minors. Moreover, though the algorithms for minor recognition have low degree polynomials as worst case bounds, their constants of proportionality are enormous, rendering them impractical for practical problems (see e.g. [Jo87a]). The general bound is $O(n^3)$ for a graph with n vertices, and even $O(n^2)$ if the family F excludes a planar graph. This is the case for the path width (see [RS83] and Proposition 3.12 below), and thus:

3.8 Theorem: *Within the class of graphs with bounded pathwidth k , k fixed, the interval graph augmentation problem can be solved in $O(n^2)$ time for a graph with n vertices.*

This result is interesting in view of the NP-hardness of the general problem when k is part of the input (see Section 4). A polynomial dynamic optimization algorithm for fixed k of order $O(n^{2k^2+4k+8})$ has also been obtained in the context of graph searching [EST87] (see also below). It is, however, open how to design a practical $O(n^2)$ algorithm.

The application of the Robertson-Seymour theory to gate matrix layout is discussed (among other problems) in a series of papers [FL87] [FL88a] [FL88b]. For a survey about computational implications of the Robertson-Seymour theory, we refer to [Jo87a].

Node searching. This problem formulation refers to a searching game on graphs introduced in [KP86] as a variant of the more investigated edge searching [Pa76].

In node searching, the edges of a graph represent a system of pipes or tunnels that are considered contaminated by a gas. The object of node searching is to clear all edges by a search. A *search* is a sequence of moves where a player places a *searcher*

(also called *guard*) on a node of the graph that carries no searcher or deletes the searcher from a guarded node.

An edge is *cleared* if both its endpoints simultaneously carry a searcher. A cleared edge may be *recontaminated* if, at a later stage of the search, there is a path from an uncleared edge to the cleared edge without any searchers on it. So in order to avoid recontamination of cleared edges, the guarded nodes must after each move form a separating set that separates the still *unsearched part* of the graph (the not yet visited vertices) from the already *searched part* (all vertices that carried a searcher in the past).

A search is called *optimal* if the maximum number of searchers on the graph at any point is as small as possible. This number is called the *node-search number* of G , and denoted by $ns(G)$.

It was shown in [KP86] that there always is an optimal search without recontamination of cleared edges. This was used in [KP85] to show the following unexpected relationships to interval graph augmentation:

3.9 Theorem: *For any graph G , $ns(G) = t(G)$.*

The proof of this theorem is based on the following ideas. If H is an interval graph augmentation of G , then any consecutive arrangement $C_1, \dots, C_k$ of the maximum cliques of H defines a search by letting the searchers move in this order through $C_1, \dots, C_k$. If C_i is guarded, then searchers from $C_i - C_{i+1}$ and possibly new searchers may be moved to $C_{i+1} - C_i$ until C_{i+1} is guarded. It is easy to see that this defines a search without recontamination with $\omega(H)$ searchers.

In the converse direction, any recontamination-free search of G assigns to every node v of G the interval $[i, j]$ whose endpoints are the first and last step in the search at which v is occupied by a searcher. (Note that this definition makes sense since the search is recontamination-free.) Since every edge is searched, the intervals assigned to its endpoints intersect. So the interval representation induces an interval graph H with $E(G) \subseteq E(H)$. The maximum number k of searchers in this search is obviously just the maximum number of pairwise intersecting intervals, i.e. $\omega(H)$.

Node searching is also closely related to other linear graph layout problems and to pebbling games on graphs [KP86]. In particular, the interpretation as progressive pebbling game gives the following useful result [KP86] for investigating or generating searches.

Consider an acyclic orientation of the edges of G and a dynamic assignment of searchers to vertices that observes the rules

(3.8) A vertex may accept a searcher only when all its immediate predecessors carry a searcher.

(3.9) Every vertex is assigned a searcher exactly once.

Then:

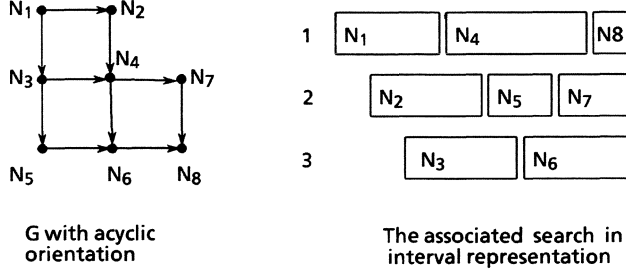


Figure 3.3. An illustration of a search

3.10 Proposition: Every assignment observing (3.8) and (3.9) defines a recontamination-free search of G , and every recontamination-free search of G can be obtained in that way.

The first part follows easily by induction and the observation that the “foremost” searchers (those that have an unvisited immediate successor) form a separating set in G that separates the searched part from the unsearched part. The other direction is obtained by considering the orientation of G defined by $u < v$ if u is visited by a searcher before v . An example is given in Figure 3.3.

We call such a search a *directed search*.

Another useful application is the combination of the search interpretation with a structural decomposition of graphs, the split decomposition [Cu82].

A *split* in an undirected graph G is a partition $V(G) = V_1 \cup V_2$ of the vertex set of G such that $|V_i| \geq 2$ ($i = 1, 2$) and

(3.10) The edges of G going from V_1 to V_2 induce a complete bipartite graph.

3.11 Lemma: Let $V(G) = V_1 \cup V_2$ be a split of G and let $A_i \subseteq V_i$ ($i = 1, 2$) be the vertices of the associated complete bipartite graph. Then every recontamination-free search of G has a step at which all vertices from A_1 or all vertices from A_2 simultaneously carry a searcher. So in particular, $ns(G) \geq \min\{|A_1|, |A_2|\}$.

This can be seen as follows. If the statement is not true, then there is a first step of the search at which a searcher is deleted from the endpoint (u , say) of an already cleared edge $(u, v) \in A_1 \times A_2$. Since neither A_1 nor A_2 are completely visited at that step, there is an uncleared edge $(x, y) \in A_1 \times A_2$. But then (u, v) is recontaminated via the path $(x, y), (y, u)$.

Still another application is the following argument from [KP86], which, in the context of edge searching, is due to [Pa76]. It also shows that the search number is unbounded on the class of trees. This gives also the missing argument for the $O(n^2)$ algorithm for bounded tree width (the class of graphs with bounded tree width excludes some trees and thus a planar graph).

3.12 Lemma: *Let G contain a vertex v of degree 3 whose deletion separates G into three connected components G_1, G_2, G_3 , each of which has node search number $ns(G_i) = t$. Then $ns(G) = t + 1$.*

It is easy to see that $t + 1$ searchers suffice. (Put a searcher on v and search G_1, G_2, G_3 with the remaining t searchers). To show that they are also necessary, assume that t searchers suffice for G .

Since t searchers are already required for each of the subgraphs G_1, G_2, G_3 , there is a moment at which one of them (G_1 , say) has already been searched, all searchers are on the second one (G_2 , say), and the last of them is still unvisited. But then recontamination takes place between G_3 and G_1 , a contradiction.

As a consequence, one obtains [KP86]:

3.13 Proposition: *The search number of a complete ternary tree T is equal to its height plus one.*

It was already mentioned that node searching is a variant of the more investigated edge searching. In edge searching, an edge is cleared by letting a searcher go through it (instead of by occupying both endpoints as in node searching). It is therefore possible [KP86] to obtain (optimal) node searchers on G from (optimal) edge searches on a slight modification of G (replace each edge of G by three parallel edges). Exploitation of this transformation and known results for edge searching on trees [MHGJP88] and dynamic programming formulations [EST87] yield a linear time search algorithm for trees (see also Section 4) and a polynomial time algorithm of order $O(n^{2k^2+4k+8})$ for graphs with search number at most k , k fixed.

Alternating paths. This final equivalence is considered in many papers on PLA folding. It views tracks in a layout of M as directed paths in the complement $\bar{G}$ of the incompatibility graph G of M . The directed edges of these paths may be considered as being added to G . This gives the following formulation.

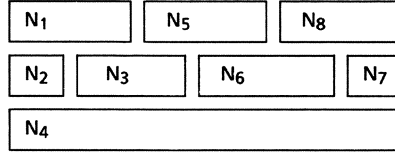
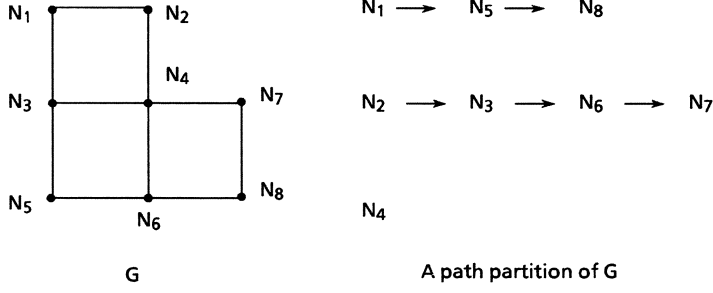
Let G be the incompatibility graph of a MPP, and consider the edges of G as colored red. A *path partition* (or *multiple folding*) of G is a set of directed green edges F such that the following two conditions are satisfied.

- (3.11) The subgraph defined by the green edges is a collection of directed paths (i.e., indegree and outdegree of every vertex is at most 1). This constraint is called the *degree constraint*.
- (3.12) There exists no cycle of alternating directed green path segments and red edges (*alternating cycle*). This constraint is called the *cycle constraint*.

The size of a path partition is the number of green paths.

3.14 Theorem: *Determining a minimum size path partition is polynomially equivalent to interval graph augmentation.*

This can be seen as follows. From a path partition with t paths, one can construct an interval representation of an interval augmentation H of G with $\omega(H) = t$ by a left to right scan through the t paths. Initially, the intervals corresponding to the



The associated interval representation
of Theorem 3.15

Figure 3.4. An illustration of a path partition

minimal vertices in the t paths are “opened”. When an interval is closed, the interval of the next vertex in the corresponding path is opened etc. Given a collection of t opened intervals, the next interval to close corresponds to a vertex u such that there are no red edges (u, w) to a vertex w that occurs after a currently open interval v on a green path containing v . Note that there is such a vertex u because of (3.12).

Since exactly t intervals are open at any moment, $\omega(H) = t$. To see that H augments G , let $(u, v) \in E(G)$ and, w.l.o.g., let u be opened before v (as intervals). Then the choice defined above ensures that u is only closed after v is opened. Hence the corresponding intervals overlap. An example of this construction is given in Figure 3.4.

The converse direction is obvious since every optimal coloring of an optimal interval graph augmentation H of G defines a path partition of size $\chi(H) = \omega(H)$. This interpretation is particularly useful for the special cases of the MPP dealing with PLA folding.

There are several other equivalent or related graph theoretic notions. We just mention vertex separation, min cut linear arrangement, bandwidth and several modifications of these problems. While vertex separation is equivalent to node search [KP86], the other notions define bounds on $t(G)$ [KP86] [Bo88].

Restrictions of the MPP. The restrictions discussed in Section 2 have natural formulations within several of the different representations of this section.

For instance, the WMPP can be modeled by fixing two cliques of G as belonging to the first and last maximal clique of the interval augmentation to construct, or by requiring the searchers to start and finish their search on specified cliques of G .

For PLA folding problems, at most two vertices may share a common track. This means in the path partition formulation that the directed green paths reduce to directed green edges. This gives the most common formulation of PLA-folding, which is due to [HNS82].

3.15 Proposition: *Let G be the incompatibility graph of a PLA folding problem. Then the minimum number of tracks for a PLA folding is equal to $|V(G)| - s$, where s is the maximum number of green directed arcs that can be added to G such that*

(3.13) *The green arcs form a matching in the complement of G (degree constraint),*

(3.14) *There is no alternating cycle of directed green edges and undirected red edges of G (cycle constraint).*

Such a set F of green arcs is called a *folding set* or simply *folding*. So Proposition 3.15 states that the PLA-folding problem is equivalent to finding a folding set of maximum size.

Another characterization can be obtained from the observation that, in an optimal layout of the associated augmented matrix M_π , the rows can be permuted in such a way that the rows with two nets appear on top and that the rightmost 1's of the first nets in these rows define a “decreasing staircase”.

This staircase pattern corresponds to the special subgraph $Z_{m,m}$ of $\bar{G}$ defined below.

A $Z_{m,m}$ (also called a *triangular clique* in [HK87]) is a bipartite graph $G = (U, V, E)$ with $U = \{a_1, \dots, a_m\}$, $V = \{b_1, \dots, b_m\}$, and $E = \{(a_i, b_j) | 1 \leq i \leq j \leq m\}$.

This gives [HK87]:

3.16 Proposition: *Finding a maximum folding set in G is equivalent to finding a maximum size $Z_{m,m}$ as (partial, not induced) subgraph of $\bar{G}$.*

Both conditions can easily be sharpened for block folding and constrained folding problems. Call a folding set F resulting in a block folding a *block folding set*. Then one obtains the following characterization of block folding, see e.g. [RL88].

3.17 Proposition: *Let G be the incompatibility graph of a PLA folding problem. Then:*

(1) *A set F of directed green arcs added to G is a block folding set iff F satisfies the degree constraint (3.13) and*

(3.15) *There is no red edge (u, v) from the head of green edge to the tail of another green edge.*

(2) *Finding a maximum block folding set in G is equivalent to finding a maximum size $K_{m,m}$ (the complete bipartite graph on $2m$ vertices) as (partial, not induced) subgraph of $\bar{G}$.*

Remember that in constrained PLA folding, the nets are preassigned to the two sides of the layout. Thus only incompatibility relations between these two sides are of importance, i.e., the incompatibility graph G can be assumed to be bipartite. Therefore, constrained PLA-folding is sometimes also called *bipartite folding* [EL84], [HK87].

The above conditions then specialize further for block folding.

3.18 Proposition: *Finding a maximum block folding in a bipartite graph G is equivalent to finding a maximum size $K_{m,m}$ as induced subgraph of the (bipartite) complement of G .*

4. Complexity Results

It was already mentioned several times that the general MPP is NP-hard. In fact, this has been obtained independently in many of the equivalent formulations, e.g. for interval graph augmentation in [KF79], for node search in [KP86], for directed node search in [Le82], and for path width in [ACP87].

The NP-hardness of the PLA folding problems (including the general MPP, but excluding block folding) is shown in [LVVS82] by a series of reductions from *matrix upper triangulation* (given an $n \times n$ 0-1 matrix A , is there a permutation of the rows and another of the columns such that resulting matrix is upper triangular?).

We will here sketch a different series of reductions that starts from constrained block folding and contains block folding and a sharper version for interval graph augmentation (i.e. the general MPP), which turns out to be NP-hard already on chordal graphs. Our starting point is (see e.g. [EL84]):

4.1 Theorem: *Constrained block folding is NP-hard.*

This follows in fact directly from Proposition 3.19 that gives the equivalence to “balanced complete bipartite subgraph” which is stated to be NP-complete in [GJ79] (the proof has appeared in [Jo87b]).

Preassignment of nets to sides can easily be enforced by adding two gates G_0, G_{n+1} that are connected to the nets from the left and right side, respectively. Then any (block) folding of the augmented problem can only fold nets incident to G_0 with nets incident to G_{n+1} , and the sides are (up to reversal of the layout) fixed by the position of G_0 and G_{n+1} . In view of Theorem 4.1, this gives:

4.2 Theorem: *Block folding is NP-hard.*

This result has been sharpened in [MW89] by a different reduction from GRAPH BISECTION. Exploiting techniques from [BCLS87], [WW89] they obtain:

4.3 Theorem: *Block folding is NP-hard even for graphs with degree at most k for any fixed $k \geq 3$.*

The reduction from block folding to IGA on chordal graphs is based on the following equivalent formulation of block folding.

Given: A graph G and an integer k .

Question: Is there a partition of $V(G)$ into three sets V_1, V_2, V_3 such that
 (i) every path from V_1 to V_3 goes through a vertex of V_2 ,
 (ii) $\min\{|V_1|, |V_3|\} \geq k$?

The answer to such an instance is obviously yes iff there exists a block folding set of G with k green edges (viz. from vertices to V_1 to vertices of V_3). Based on this formulation, the following result is obtained in [Gu89].

4.4 Theorem: *Gate matrix layout and pathwidth are already NP-hard on the class of chordal graphs.*

Recall that a graph is *chordal* (or *triangulated*) if every elementary cycle $v_1, v_2, \dots, v_k, v_1$ of length $k \geq 4$ possesses a chord, i.e. an edge (v_i, v_j) with $1 \leq i < j + 1 \leq k + 1$.

Chordal graphs form a natural generalization of interval graphs, see e.g. [Go80] for more information about chordal graphs. The chordal graphs needed in the proof are quite special. They consist of a set of maximal cliques that overlap in a central clique.

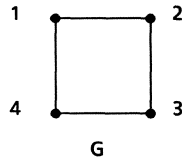
In more detail, let G, k be an instance of the above formulation of block folding such that G is w.l.o.g. connected. From G we construct such a special chordal graph H as follows. H contains a maximal clique C_0 (called the *central clique*) with vertex set $V(G)$. For each edge $(u, v) \in E(G)$, a maximal clique C_{uv} is added that consists of the vertices $u, v \in C_0$ and $|V(G)|$ additional vertices that are incident only to vertices in C_{uv} . Clearly, this graph is of the desired type.

From H one can construct a net-gate matrix M with incompatibility graph H by introducing a gate for each of the maximal cliques of H . Consider an augmented matrix M_π of M and let C^+ and C^- be the cliques (gates) before and after the central clique (gate) C_0 in M_π . Since every net corresponding to an original vertex of G that is incident to some gate from C^+ or C^- is also incident to C_0 , the order of the gates in C^+ or C^- does not influence the number of tracks.

Let V^+ and V^- denote the nets of C_0 (vertices of G) incident to a gate from C^+ and C^- , respectively, and let $V_1 := V^+ - V^-$, $V_2 := V^+ \cap V^-$ and $V_3 := V^- - V^+$. It can then be shown that V_1, V_2, V_3 form a partition of $V(G)$ with the above disconnecting property, and that $\min\{|V_1|, |V_3|\} \geq k$ is equivalent to $t(M) \leq 2 \cdot |V(G)| - k$.

Conversely, every partition V_1, V_2, V_3 of G with the above properties can be transformed into an augmented matrix M_π by first taking all gates (cliques) C_{uv} with $u, v \in V_1 \cup V_2$ (in any order), followed by C_0 and all cliques C_{uv} with $u, v \in V_2 \cup V_3$. This proves the theorem. An example of this construction is given in Figure 4.1.

Note that if all maximal cliques that intersect the central clique are mutually disjoint, then the problem can be solved in $O(|V(G)|^2)$ time by a dynamic programming algorithm [Gu89]. This confirms that the borderline between easy and hard problems for subclasses of chordal graphs G depends essentially on the overlapping behavior of the maximal cliques of G (see also [ACP87]).



$$V_1 = \{1\}$$

$$V_2 = \{2, 4\}$$

$$V_3 = \{3\}$$

An associated partition
 V_1, V_2, V_3 with $k = 1$

	C_0	C_{12}	C_{14}	C_{23}	C_{34}
1	1	1	1		
2	1	1		1	
3	1			1	1
4	1		1		1
5		1			
6		1			
7		1			
8		1			
9			1		
10			1		
11			1		
12			1		
13				1	
14				1	
15				1	
16				1	
17					1
18					1
19					1
20					1

The net gate-matrix of H

	C_{12}	C_{14}	C_0	C_{23}	C_{34}
1	1	1	1		
2	1	*	1	1	
3			1	1	1
4		1	1	*	1
5	1				
6	1				
7	1				
8	1				
9		1			
10		1			
11		1			
12		1			
13				1	
14				1	
15				1	
16				1	
17					1
18					1
19					1
20					1

The augmented matrix corresponding
to V_1, V_2, V_3

Figure 4.1. An illustration of the proof of Theorem 4.4

A reduction from gate matrix layout to constrained PLA-folding can be obtained by turning an arbitrary graph G into a bipartite graph $H = (U, V, E)$ as follows:

$U := V(G)$, $V := \{v' | v \in V(G)\}$ (a copy of $V(G)$), and $(u, v') \in E$ (with $u \in U$ and $v' \in V$) iff $(u, v) \in E(G)$ or $u = v$.

Then any constrained folding set F for H corresponds uniquely to a collection of tracks for G by combining the edges of F to paths (tracks) (x_1, x'_2) , (x_2, x'_3) , (x_3, x'_4) , $\dots$, (k'_k, x_1) . Then $t(G) = |V(G)| - |F|$, i.e. maximizing $|F|$ in H corresponds to minimizing $t(G)$ in G . Hence:

4.5. Theorem: *Constrained PLA folding is NP-hard.*

Finally, the reduction from constrained PLA folding to PLA folding is achieved in the same way as from constrained block folding to block folding.

4.6. Theorem: *PLA folding is NP-hard.*

There are several other NP-hardness results related to gate matrix layout and PLA folding. We mentioned already the directed search problem [Le82]. Another such problem is

Orderability:

Given: A graph G of red edges, a set F of green edges of $\bar{G}$.

Question: Is there an orientation of the edges of F such that F is a folding?

Orderability is shown to be NP-complete in [HNS82]. It is solvable in polynomial time for constrained PLA folding [Ra88].

Other variants of the PLA-folding problem not discussed here are shown to be NP-hard in [ALN88].

We consider now some special classes of graphs on which the gate matrix layout problem can be solved in polynomial time. Most of the arguments leading to polynomial algorithms come from node searching (in particular Lemma 3.11 and Lemma 3.12) and demonstrate again the usefulness of this interpretation.

We will start with the class of trees. As mentioned before, the polynomial algorithm for edge searching on trees [MHGJP88] can be transformed by the principles of [KP86] to a polynomial algorithm for node searching on trees. This requires $O(n)$ time for determining $ns(G)$, and $O(n \log n)$ time for finding the associated search. We sketch here a different, equally fast algorithm with a much simpler correctness proof.

The algorithm peels the tree, i.e. it starts with the leaves and works its way towards the “center” of the tree. So at a typical step of the algorithm, certain subtrees $T_1, \dots, T_r$ of the tree T have already been investigated. Each of these trees T_i has a vertex v_i connecting it to the still unsearched part of T .

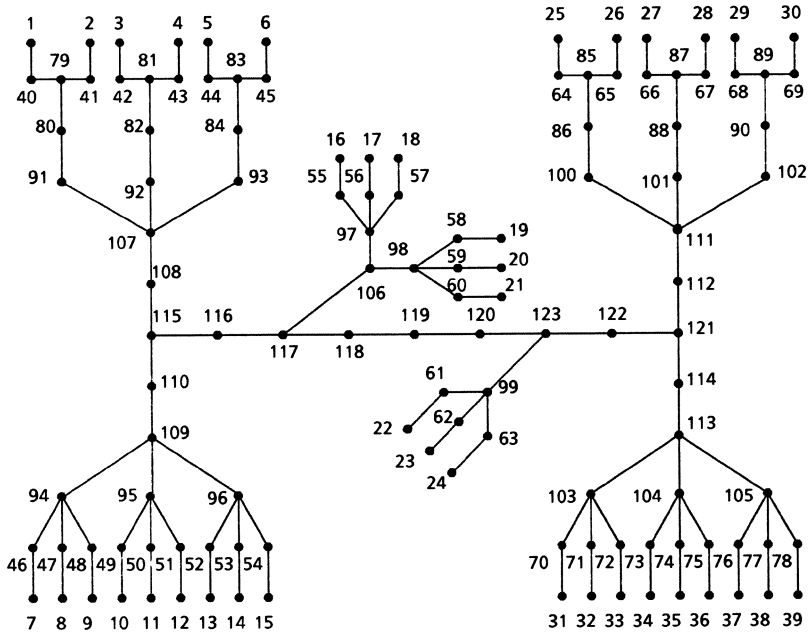
The peeling is done in *phases* $t = 1, 2, \dots, ns(T)$. At the beginning of phase t , the vertices v_i are the leaves of the remaining tree, and every T_i requires t searchers, while every $T_i - v_i$ can be searched with $t - 1$ searchers. We then peel the remaining tree starting from the v_i until we reach new vertices u_j in which the search number must go up to $t + 1$ (or T has been searched completely). The current phase is completed when all v_i have been processed and are connected to some u_j in the already searched part of the tree.

All vertices investigated in phase t (except the u_j which are the starting vertices for the next phase) receive the *label* $label(v) = t$, $t = 1, \dots, ns(T)$. These labels have the following important property:

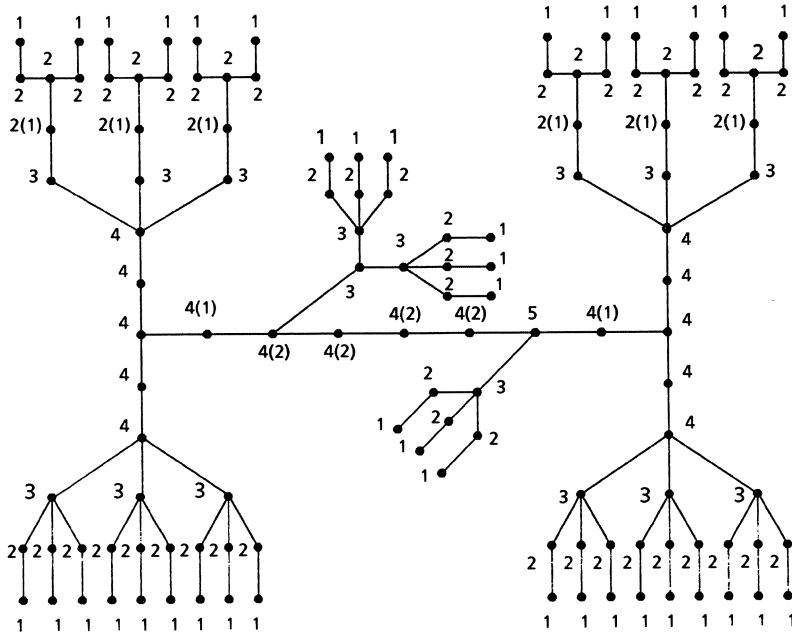
Let, for a vertex with label t , T_v be the connected component in the subgraph of all vertices with label at most t that contains v . Then:

$$(4.1) \quad ns(T_v) = label(v).$$

An example of the peeling and the different phases is given in Figure 4.2.



a) Processing order of the vertices



b) Pase labels with respect to the processing order of a)

Figure 4.2. An example tree for the algorithm

The essential part of the algorithm consists in identifying the new vertices u_j in which the search number must go up. There are several special cases for small t (e.g. the transition from vertex 80 to 91 in Figure 4.2), but the general procedure is as follows.

We first explore from every v_i the path (in the remaining tree) connecting it to the still unexplored part of T until we reach a vertex v (where $v = v_i$ is possible) with 2 unexplored neighbors or another neighbor u with $\text{label}(u) = \text{label}(v_i)$. In Figure 4.2, such paths are e.g. 109, 110 or 111, 112. After this *path exploration*, they are three cases.

We reach a vertex v with two unexplored neighbors. Due to the algorithm, these neighbors are connected to other subtrees with search number t , and so the search number must go up to $t + 1$ because of Lemma 3.12. Then v will be considered in a later phase. In Figure 4.2, $v = 99$ is such a vertex.

Three or more paths meet in a vertex u . Then the search number goes up in u to $t + 1$ because of Lemma 3.12. In Figure 4.2, this is e.g. the case for $u = 107$ (from 91, 92, 93) and $u = 111$ (from 100, 101, 102).

Exactly two paths next meet in a vertex u . Then the search can be carried over with t searchers from one path to the other with u in the “interior” of the corresponding layout (i.e. there is no search with t searchers that ends in u). It may, however, still be possible to search part of the tree starting in u with at most $t - 1$ searchers while a searcher guards the vertex u . To this end, we consider the unexplored neighbor of u as a leaf and start a search from it to the unexplored part until we reach the first vertex where t searchers are required. This vertex is then a leaf for the next phase. We call this situation a *fork*.

In Figure 4.2, such forks are given by $u = 106$, $u = 115$, and $u = 121$. In $u = 106$, the search stops immediately because 117 has two unexplored neighbors. In $u = 115$, we start a search along the path 116, 117, etc., where 116 is treated as a leaf. The new phase labels of this search are given in brackets. Note that the fork in 106 influences this search since a guard must remain on 106, thus implying that only two searchers are available along the path 117, 118, etc. This is why the search stops in 120. The formal stopping argument is that the subtree induced by $\{116, 117, \dots, 120, 123\}$ requires 4 searchers, which together with the fork at 115 and Lemma 3.12 increases the search number to 5.

The fork of 121 then meets the fork of 115 in 123 and brings the search number up to 5 because of Lemma 3.12 (every fork contributes two subgraphs of search number t).

These are the main ingredients of the algorithm. There are several additional remarks to be made.

The argument for an increase of the search number is always Lemma 3.12. This is in fact due to the “converse” of Lemma 3.12 [Pa76] stating that, for any tree T , $ns(T) \geq t + 1$ iff there is a vertex v at which there are three or more subtrees with search number t or more.

The path exploration in forks depends of course on the processing order of the vertices. For instance, exploring the fork in 121 first would explore the path until vertex 117.

The phrase labels and the information about forks (i.e. which paths are joined) can be directly used to obtain an optimal search. This is demonstrated in Figure 4.3 for the example of Figure 4.2.

Altogether, this gives:

4.7 Theorem: *The track number $t(T)$ of a tree T and an optimal layout can be obtained in $O(|V(T)|)$ time.*

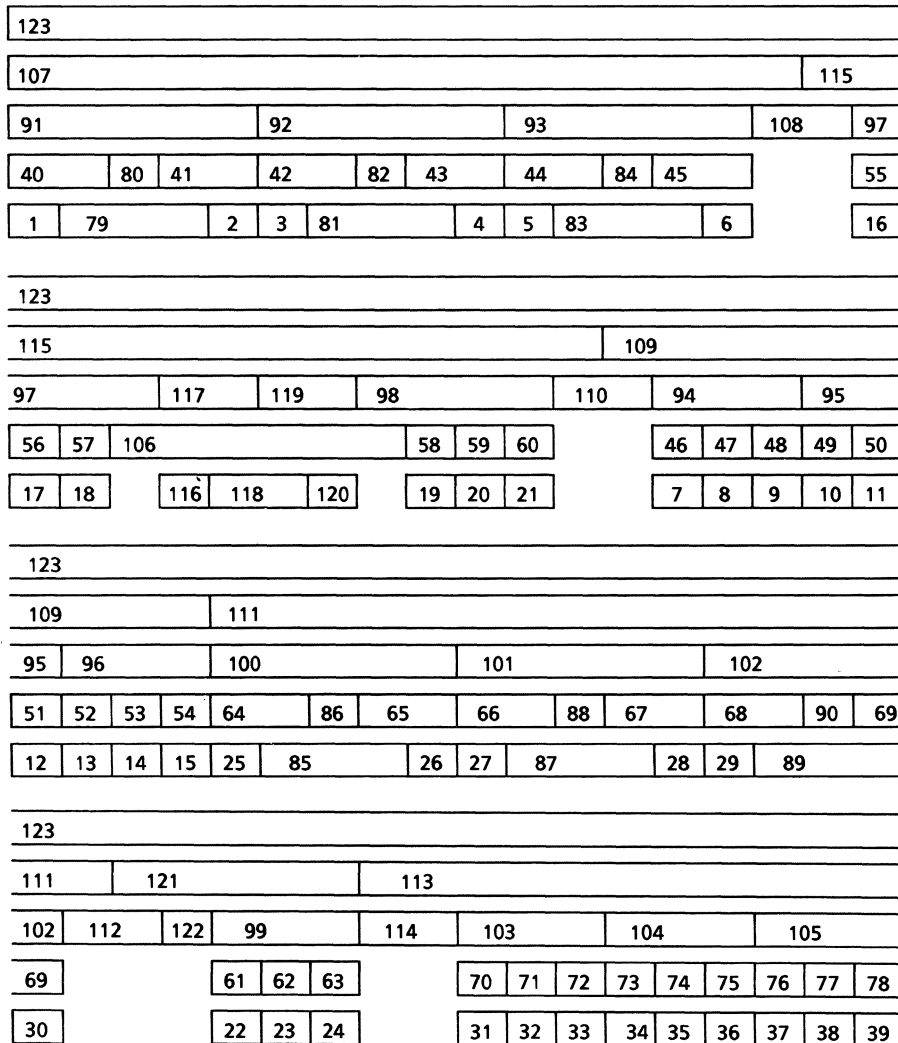


Figure 4.3. The layout corresponding to the search of Figure 4.2

The techniques underlying the tree search algorithm seem to be valid also for other “tree-structured” graphs such as (2-connected) outerplanar graphs, k -trees, and possibly partial k -trees. This is currently being investigated together with J. Gustedt and R. Müller from the TU Berlin.

Lemma 3.11 can be used to obtain a polynomial algorithm for cographs. Recall that a *cograph* (or *complement reducible graph*, see [CLS81] for details) can be defined recursively by

(4.2) The one-vertex graph is a cograph

(4.3) If $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are cographs on disjoint sets V_1, V_2 , then

$$\begin{aligned} G_1 + G_2 &:= (V_1 \cup V_2, E_1 \cup E_2) \quad \text{and} \\ G_1 * G_2 &:= (V_1 \cup V_2, E_1 \cup E_2 \cup (V_1 \times V_2)) \end{aligned}$$

are also cographs.

Then Lemma 3.11 gives:

$$(4.4) \quad ns(G_1 + G_2) = \max\{ns(G_1), ns(G_2)\}$$

$$(4.5) \quad ns(G_1 * G_2) = \min\{|V_1| + ns(G_2), |V_2| + ns(G_1)\}$$

An associated interval representation of an optimal search can be defined similarly as in the tree algorithm by defining offsets in each operation. So if a sequence of operations “*” and “+” producing the whole graph is given (a parse tree defining such a sequence for G can be obtained in $O(|V(G)| + |E(G)|)$ time [CPS85]) one obtains:

4.8 Theorem: *The track number of a cograph G (given in “decomposed” form or by its parse tree) and an optimal layout can be obtained in $O(|V(G)|)$ time.*

The operation “*” can be seen as a special split in which (with the terminology of Lemma 3.11) $A = V_1$ and $B = V_2$. It is still open whether there is a polynomial time search algorithm for the larger class of graphs that can be recursively decomposed by splits. They are known as *distance-hereditary graphs* [BM86] or *completely separable graphs* [HM87], and contain both the class of trees and the class of cographs.

Finally, we mention the class of chordal graphs discussed in connection with Theorem 4.4. By using a dynamic programming approach similar to that for PARTITION in [GJ79], [Gu89] obtains:

4.9 Theorem: *If the maximal cliques $C_0, C_1, \dots, C_m$ of G fulfill*

$$(4.6) \quad C_i \cap C_j \neq \emptyset \Rightarrow C_i \cap C_j = C_i \cap C_0,$$

then $t(G)$ and an optimal layout can be obtained in $O(|V(G)|^2)$ time.

Condition (4.6) means that the cliques $C_1, \dots, C_n$ have a special overlap structure with the “central” clique C_0 . Similar arguments yield also a polynomial algorithm for *split graphs* [Go80], i.e. those chordal graphs whose complement is also chordal.

For PLA-folding problems, much less is known about polynomially solvable cases. An algorithm for constrained PLA-folding on trees has been obtained in [HK87]. Results of [Bo87] on the balanced complete bipartite subgraph problem (in particular the transformation applied there) show, when combined with Propositions 3.16 and 3.18, the polynomial solvability of constrained block folding and block folding on partial k -trees. It is easy to also obtain polynomial time dynamic programming algorithms for these cases.

5. Algorithms

Due to the VLSI-background, many algorithms have been proposed and studied in the literature. The majority of them can be classified as (sometimes a combination of) heuristics, branch-and-bound algorithms, or dynamic programming algorithms.

Representatives for the different technologies discussed here are [YKK75], [As82] for Weinberger arrays, [LVVS82], [Li83], [WHW85], [Leo86], [NFKY86], [DKL87] for gate matrix layout, and [LL84], [HDB86], [HK87], [KH87], [ALN88] for PLA folding (see [GL88] for additional references). The problems studied in these papers are usually not larger than 50×60 (in terms of the net-gate matrix), with the exception of 100×80 in [ALN88].

Only little is known about the performance of heuristics for these problems. By using standard arguments from [GJ79], the existence of an approximation algorithm for gate matrix layout with a constant absolute performance guarantee is ruled out in [DKL87]:

5.1 Theorem: *Unless $P = NP$, there is no approximation algorithm A for gate matrix layout with*

$$(5.1) \quad A(I) \leq OPT(I) + K, \quad K \in \mathbb{N} \text{ fixed}$$

for all instances I .

It is well known [GJ79] that this also rules out the existence of a fully polynomial approximation scheme for gate matrix layout.

The existence of approximation algorithms with a *constant relative* performance guarantee is open. There are, however, some indications that they might not exist.

For PLA-folding problems, such an indication is given in [RL88]. Call two problems *equivalent with respect to approximation* if both or none are approximable with constant relative performance guarantee in polynomial time. Then [RL88]:

5.2 Theorem: *PLA-folding, block folding, and constrained block folding are equivalent with respect to approximation, when viewed as maximization problems (maximize the size of a folding).*

The proof is based on transforming feasible solutions of one problem to feasible solutions of the other while preserving a constant relative performance guarantee. For instance, let algorithm A produce, for an instance I of PLA-folding, a solution

with $A(I)$ folded pairs such that $A(I) \leq K \cdot OPT(I)$, where $OPT(I)$ denotes the size of an optimal folding. Then we can transform this solution to a solution of block folding, with size $A'(I)$ by taking, in a decreasing staircase arrangement of the layout (cf. the remarks preceding Proposition 3.17), the first nets of rows $1, 2, \dots, \lfloor A(I)/2 \rfloor$ and fold them with the last nets of rows $\lfloor A(I)/2 \rfloor + 1, \dots, A(I)$. Clearly, this gives a block-folding with $A'(I) = A(I)/2$ folded nets, and so the maximum size $OPT'(I)$ of a block folding fulfills $OPT'(I) \leq OPT(I) \leq K \cdot A(I) = 2K \cdot A'(I)$.

This equivalence result is combined in [RL88] with the following, unexpected result that provides the indication for non-approximability.

5.3 Theorem: *If there is an approximation algorithm A for block folding (in the maximization version) with relative performance guarantee K , $K \in \mathbb{N}$ fixed, then there is also one with relative performance guarantee $1 + \varepsilon$ for every fixed $\varepsilon > 0$.*

Note, however, that these results carry only through for the maximization version of the PLA-folding problems (maximize the size of a folding set). Both proofs fail for the—perhaps more natural—minimization of the number of tracks required in a layout.

Current work at the TU Berlin applies Lagrangean relaxation and subgradient methods combined with branch-and-bound techniques to the constrained block folding problem [Mü89]. Starting point is the formulation as a matching problem on a bipartite graph with side constraints (Proposition 3.17). Relaxation of the side constraints (3.15) and an appropriate reformulation then leads to a special well-solved min-cost flow problem whose use in branch-and-bound schemes seems to be promising.

For gate matrix layout, an indication for the non-existence of approximation algorithms with constant relative performance guarantee is obtained in [DKL87] by considering algorithms that are *on-line* with respect to the nets. This means that the nets are processed in an incremental fashion according to the following rules:

- (1) A partial layout for the nets processed so far has already been constructed and may not be changed when the next net is processed.
- (2) The next net N_i to be processed is chosen such that its addition to the partial layout causes the least increase in the number of tracks.

By designing a class of examples, [DKL87] show that such on-line algorithms cannot guarantee a constant relative performance error.

This definition of on-line seems, however, to be very restrictive, since the algorithms even fail to construct an optimal layout for interval graphs. (In fact, the class of examples of [DKL87] consists entirely of interval graphs.) The reason for this behavior is the rigidity of the already constructed partial layout.

We suggest here a less rigid approach that is still on-line and equally fast, but allows more flexibility in modifying the partial layout. The basic idea is to maintain not a partial layout, but the corresponding interval graph (see Section 3), and to represent it by its MPQ-tree.

The MPQ-tree (see [KM89] for details) is a data structure that represents an interval graph H and all associated interval orders (cf. Lemma 3.1) in $O(|V(H)|)$ space. It permits also fast updating when a vertex is added to H . Such an update will always recognize when $H + v$ is again an interval graph and modify the MPQ-tree accordingly. (So the examples from [DKL87] are solved optimally.) If $H + v$ is not an interval graph, then there are several possibilities to augment $H + v$ to an interval graph H^* by considering different interval orders associated with H . So keeping the maximum clique size small then means to permute or invert the nodes of the MPQ-tree representing H in such a way that adding v increases $\omega(H)$ as little as possible. This “local” optimization can be done in polynomial time.

Altogether, this leads to a class of on-line algorithms based on incremental interval graph generation by means of MPQ-trees. Current work at the TU Berlin in this direction seems to be quite promising.

Acknowledgement

I would like to thank H. L. Bodländer, A. Proskurowski, and M. M. Syslo for various discussions and many helpful remarks.

References

- [ALN88] C. Arbib, M. Lucertini, and N. Nicoloso (1988). Optimal design of programmed logic arrays, preprint, Università degli Studi di Roma “La Sapienza”.
- [ACP87] S. Arnborg, D. G. Corneil, and A. Proskurowski (1987). Complexity of finding embeddings in a k -tree, *SIAM J. Alg. Disc. Meth.* 8, 277–284.
- [As82] Asano, T. (1982). An optimum gate placement algorithm for MOS one-dimensional arrays, *Journal of Digital Systems*, 1–27.
- [Bo87] H. L. Bodländer (1987). Dynamic programming on graphs with bounded tree width. Report RUU-CS-87-22, University of Utrecht.
- [BM86] H. J. Bandelt and H. M. Mulder (1986). Distance-hereditary graphs, *J. Comb. Th. B* 41, 182–208.
- [Bo88] H. L. Bodländer (1988). Some classes of graphs with bounded treewidth, *Bulletin EATCS* 36, 116–125.
- [BL76] S. Booth and S. Lueker (1976). Testing for the consecutive ones property, interval graphs, and planarity using PQ-tree algorithms, *J. Comput. Syst. Sci.* 13, 335–379.
- [BMHS84] R. K. Brayton, C. McMullen, G. D. Hachtel, and A. Sangiovanni-Vincentelli (1984). *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer Academic Publishers, Boston, MA.
- [BCLS87] T. N. Bui, S. Chaudhuri, F. T. Leighton and M. Sipser (1987). Graph bisection algorithms with good average case behavior, *Combinatorica* 7, 171–191.
- [CLS81] D. G. Corneil, H. Lerchs, and L. Stewart Burlingham (1981). Complement reducible graphs, *Discrete Appl. Math.* 3, 163–174.
- [CPS85] D. G. Corneil, Y. Pearl, and L. Stewart (1985). A linear recognition algorithm for cographs, *SIAM J. Computing* 14, 926–934.
- [Cu82] W. H. Cunningham (1982). Decomposition of directed graphs, *SIAM J. Alg. Discr. Methods* 3, 214–228.
- [DKL87] N. Deo, M. S. Krishnamoorthy and M. A. Langston (1987). Exact and approximate solutions for the gate matrix layout problem, *IEEE Trans. on Computer-Aided Design* 6, 79–84.
- [EL84] J. R. Egan and C. L. Liu (1984). Bipartite folding and partitioning of a PLA, *IEEE Trans. CAD* 3, 191–199.
- [EST87] J. Ellis, I. Sudborough and J. Turner (1987). Graph separation and search number, Report DCS-66-IR, University of Victoria

- [Ev79] S. Even (1979). *Graph Algorithms*, Computer Science Press, Potomac, MD.
- [FL87] M. R. Fellows and M. A. Langston (1987). Nonconstructive advances in polynomial-time complexity. *Infor. Proc. Letters* 26, 157–162.
- [FL88a] M. R. Fellows and M. A. Langston (1988). Nonconstructive tools for proving polynomial-time decidability, *J. ACM* 35, 727–739.
- [FL88b] M. R. Fellows and M. A. Langston (1988). Layout permutation problems and well-partially-ordered sets. *Proc. 5th MIT Conf. on Advanced Research in VLSI*, 315–327.
- [FM75] H. Fleisher and L. I. Maissel (1975). An introduction to array logic, *IBM J. Res. and Developm.* 19, 98–109.
- [FRS87] H. Friedman, N. Robertson and P. D. Seymour (1987). The methamathematics of the graph minor theorem, in “Applications of Logic to Combinatorics”, AMS Contemporary Mathematics Series, Amer. Math. Soc. Providence, R. I., to appear.
- [FG65] D. R. Fulkerson and O. A. Gross (1965). Incidence matrices and interval graphs, *Pacific J. Math.* 15, 835–855.
- [GL88] D. G. Gajski and Y.-L. S. Lin (1988). Module generation and silicon compilation, in B. Preas and M. Lorenzetti (eds.) *Physical Design Automation of VLSI Systems*, Benjamin/Cummings, Menlo Park, CA, 283–345.
- [GJ79] M. R. Garey and D. S. Johnson (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco.
- [Go80] M. C. Golumbic (1980). *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, New York.
- [Go85] M. C. Golumbic (1985). Interval graphs and related topics, *Discrete Math.* 55, 113–121.
- [GLL82] U. I. Gupta, D. T. Lee and I. Y.-T. Leung (1982). Efficient algorithms for interval graphs and circular arc graphs, *Networks* 12, 459–467.
- [Gu89] J. Gustedt (1989). Path width for chordal graphs is NP-complete, preprint, TU Berlin.
- [HNS82] G. D. Hachtel, A. R. Newton, and A. L. Sangiovanni-Vincentelli (1982). An algorithm for optimal PLA folding, *IEEE Trans. on CAD of Integrated Circuits and Systems*, CAD-1 (2), 63–77.
- [HM87] P. L. Hammer and F. Maffray (1987). Completely separable graphs, preprint, Rutgers University.
- [HS71] A. Hashimoto, and J. Stevens (1971). Wire routing by optimizing channel assignment within large apertures, *Proc. of 8th Design Automation Conf.*, 155–169.
- [HK87] T. C. Hu and Y. S. Kuo (1987). Graph folding and programmable logic array, *Networks* 17, 19–37.
- [HDB86] S. Y. Hwang, R. W. Dutton, and T. Blank (1986). A best-first search algorithm for optimal PLA folding, *IEE Trans. on CAD*, CAD-5 (3), 433–442.
- [Jo87a] D. S. Johnson (1987). The NP-completeness column: An ongoing guide, *J. Algorithms* 8, 285–203.
- [Jo87b] D. S. Johnson (1987). The NP-completeness column: An ongoing guide, *J. Algorithms* 8, 438–448.
- [KF79] T. Kashiwabara and T. Fujisawa (1979). NP-completeness of the problem of finding a minimum-clique-number interval graph containing a given graph as a subgraph, *Proc. 1979 Intern. Symposium on Circuits and Systems*, 657–660.
- [KP85] L. M. Kirousis and C. H. Papadimitriou (1985). Interval graphs and searching, *Discrete Math.* 55, 181–184.
- [KP86] L. M. Kirousis and C. H. Papadimitriou (1986). Searching and pebbling, *Th. Comp. Science* 47, 205–218.
- [KM89] N. Korte and R. H. Möhring (1989). An incremental linear-time algorithm to recognize interval graphs, *SIAM J. Computing* 18, 68–81.
- [KH87] Y. S. Kuo and T. C. Hu (1987). An effective algorithm for optimal PLA column folding INTEGRATION, the VLSI journal 5, 217–230.
- [Le82] T. Lengauer (1981). Black-white pebbles and graph separation, *Acta Inf.* 16, 465–475.
- [Leo86] H. W. Leong (1986). A new algorithm for gate matrix layout. *Digest, Intl. Conf. on Computer-Aided Design*, 316–319.
- [LL84] J. L. Lewandowski and C. L. Liu (1984). A branch and bound algorithm for optimal PLA folding. *Proc. of the 21st Design Automation Conf.*, 425–433.
- [Li83] J. T. Li (1983). Algorithms for gate matrix layout, *Proc. 1983 IEEE Int. Symp. Circuits and Systems*, 1013–1016.
- [LLa80] A. Lopez and H. Law (1980). A dense gate matrix layout method for MOS VLSI, *IEEE Trans. on Electronic Devices*, ED-27 (8), 1671–1675.

- [LVVS82] M. Luby, U. Vazirani, V. Vazirani, and A. L. Sangiovanni-Vincentelli (1982). Some theoretical results on the optimal PLA folding problem, *IEEE Internat. Confer. Circuits and Systems*, 165–170.
- [MS89] F. Makedon and I. H. Sudborough (1989). On minimizing width in linear layouts, *Discrete Appl. Math.* 23, 201–298.
- [MC80] C. Mead and L. Conway (1980). *Introduction of VLSI Systems*, Addison Wesley, Reading MA.
- [MHGJP88] N. Megiddo, S. L. Hakimi, M. R. Garey, D. S. Johnson, and C. H. Papadimitriou (1988). The complexity of searching a graph, *J. ACM* 35, 18–44.
- [Mö85] R. H. Möhring (1985). Algorithmic aspects of comparability graphs and interval graphs, in I. Rival (ed.) *Graphs and Order*, Reidel, Dordrecht, 41–101.
- [Mö89] R. H. Möhring (1989). Computationally tractable classes of ordered sets, in I. Rival (ed.) *Algorithms and Order*, Kluwer Acad. Publ., Dordrecht, 105–194.
- [Mü89] R. Müller (1989), in preparation.
- [MW89] R. Müller and D. Wagner (1989). α -vertex separation is NP-complete for 3-regular graphs, preprint, TU Berlin.
- [NFKY86] K. Nakatani, T. Fujii, T. Kikuno, and N. Yoshita (1986). A heuristic algorithm for gate matrix layout, *Digest Intl. Conf. on Computer-Aided Design*, 324–327.
- [OMKK79] T. Ohtsuki, H. Mori, E. S. Kuh, T. Kashiwabara, and T. Fujisawa (1979). One-dimensional logic gate assignment and interval graphs, *IEEE Trans. on Circuits and Systems*, 675–684.
- [Pa76] T. D. Parsons (1976). Pursuit-evasion in a graph, in Y. Alavi and D. Lick (eds.), *Theory and Applications of Graphs*, Springer Verlag, Berlin, 426–441.
- [Ra88] S. S. Ravi (1988). A note on the orderability problem for PLA folding, to appear in *Discrete Appl. Math.*
- [RL88] S. S. Ravi and E. L. Loyd (1988). The complexity of near-optimal programmable logic array folding, *SIAM J. Computing* 17, 696–710.
- [RS83] N. Robertson and P. D. Seymour (1983). Graph minors I. Excluding a forest, *J. Comb. Th. B.* 35, 39–61.
- [RS86] N. Robertson and P. D. Seymour (1986). Graph minors XIII. The disjoint paths problem, preprint.
- [RS87] N. Robertson and P. D. Seymour (1987). Graph Minors XVI. Wagner's conjecture, preprint.
- [SM83] K. H. Schmidt and K. D. Mueller-Glaser (1983). NMOS dense gate matrix VLSI Design, *IEEE J. of Solid-State Circuits*, SC-18, 157–159.
- [WW89] D. Wagner and F. Wagner (1989). Graph separation is NP-hard even for graphs with bounded degree, preprint TU Berlin.
- [Wei67] A. Weinberger (1967). Large scale integration of MOS complex logic: a layout method, *IEEE Journal of Solid-State Circuits*, SC-2 (4), 182–190.
- [Win82] O. Wing (1982). Automated gate matrix layout, *Intl. Symposium on Circuits and Systems*, 681–685.
- [Win83] O. Wing (1983). Interval-graph-based circuit layout, *Proc. IEEE 1983, Int. Conf. on CAD*, 84–85.
- [WHW85] O. Wing, S. Huang, and R. Wang (1985) Gate matrix layout, *IEEE Trans. on CAD*, CAD-4 (3), 220–231.
- [YKK75] H. Yoshizawa, H. Kawanishi, and K. Kami (1975). A heuristic procedure for ordering MOS arrays, *Proc. 12th Design Automation Conference*, 384–389.

Rolf H. Möhring
 Technical University of Berlin
 Fachbereich Mathematik
 Strasse des 17. Juni 136
 D-1000 Berlin